

Lezione (3) 21/09/2022

martedì 20 settembre 2022

09:34

Linguaggi di programmazione - teoria (sintassi)

Prob. Computazionale = problema da risolvere con calcolatore --> problem solving
se realmente risolve il problema + analisi efficienza algoritmo [velocità esecuzione]
all'agente di calcolo (esemplificazione del modello calcolatore) --> linguaggio di programmazione
comprendere in modo univoco, chiaro e non ambiguo

=> analisi della correttezza delle istruzioni scritte nel linguaggio (formalizzazione della sintassi)

Linguaggio:

- **lessico** = come si scrivono le parole, ci permette di capire se una parola è scritta correttamente
=>err. Ortografici)
- **grammatica** => regole per comporre le frasi corrette da un punto di vista grammaticale
(manguia Pippo mela / 3+-4 errata perché non segue lo schema soggetto, verbo, complemento)
- **semantica** => indaga il significato delle frasi in base alle associazioni delle parole
costituiscono (la mela mangia Pippo è corretta dal punto di vista lessicale e grammaticale ma
priva di significato reale)

Lessico (scanner)

=> insieme delle parole di un linguaggio --> le parole vengono costruite in base ai caratteri
per la costruzione delle frasi (il lessico della ling. Ita. È costituito dall'insieme delle parole
linguaggio C/js = insieme delle parole formate dai caratteri ASCII)

Sintassi (parser)

assi)

e analisi --> algoritmo (+ analisi correttezza algoritmo, operazioni]) --> l'algoritmo deve essere comunicato
programmazione (ling. Che la macchina è in grado di

el linguaggio)

correttamente o meno (Pippo mangia la mla / 3+ 4l

maticale (non corrette dal punto del significato)
omplemento)

e del linguaggio e gli **oggetti concreti** che esse
maticale, ma non è corretta la sua semantica, poiché è

caratteri dell'alfabeto --> le parole vengono utilizzate
parole trovabili nel vocabolario ita. / lessico del

=> regole (grammaticali) per mettere insieme le parole di un linguaggio e formare frasi (espressioni) (espressioni = 3 corretta sintatticamente ma priva di semantica)

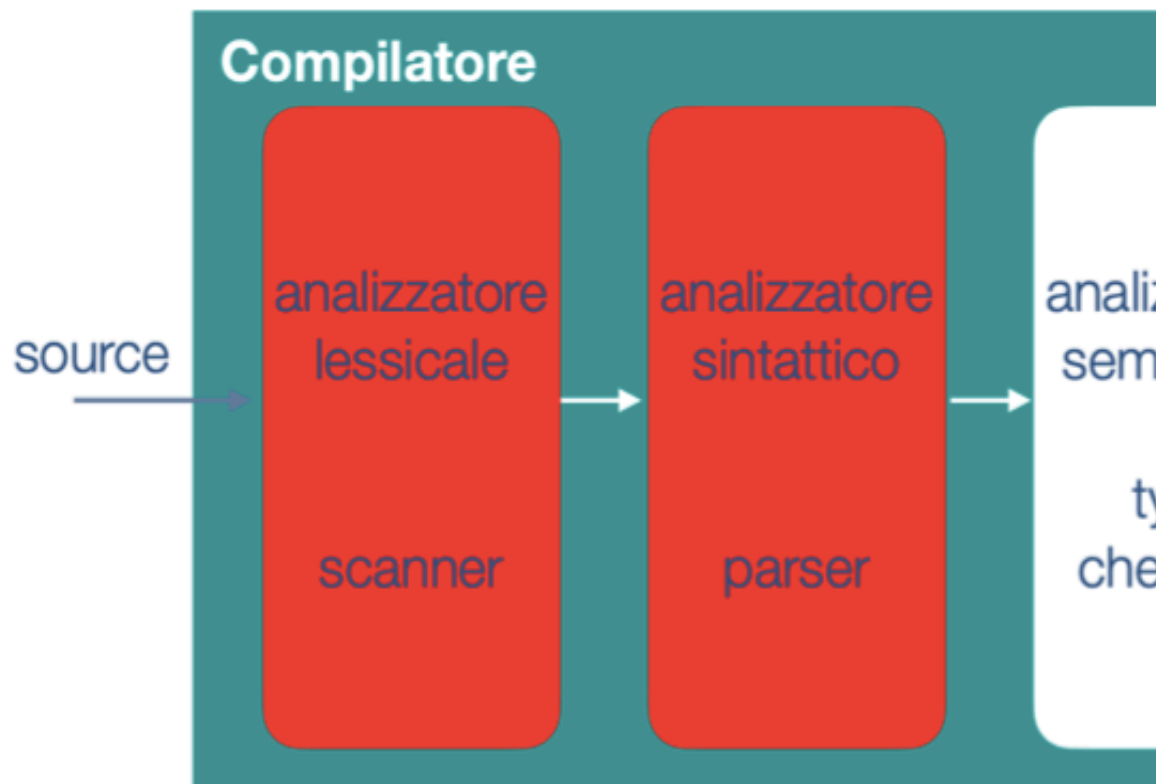
Semantica (type checking)

=> significato delle frasi /fornisce gli strumenti per definire il significato (associazioni di parole a azioni reali della macchina)

=>analisi fatta unicamente dopo l'analisi lessicale e sintattica

=>dopo l'analisi semantica viene generato il codice oggetto (l'algoritmo può essere ottimizzato)

la sintassi non considera il significato delle parole
legale (corretta) secondo le regole della grammatica



Sintassi (parser)

=>necessità di un linguaggio composto da un insieme di simboli atomici (caratteri) (le parole)

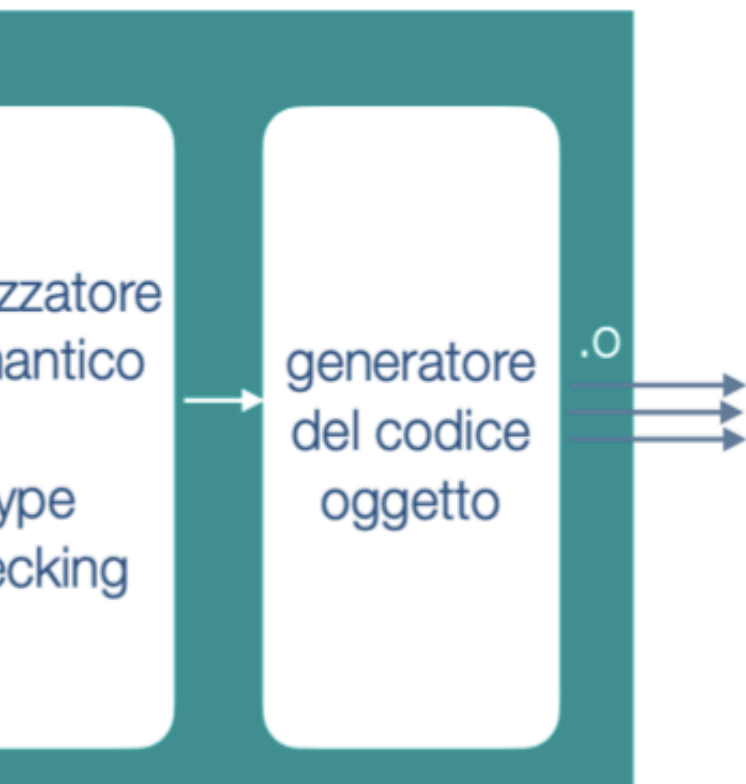
Concatenazione

delle frasi corrette (sintatticamente) (int a [10], a [12]

ni tra ogg. reali e parole corrispondenti / tra istruzioni

mandato in esecuzione)

e per determinare se una frase è
tica del linguaggio



i) che vengono concatenati per formare delle sequenze

Concatenazione

operazione definita sui simboli di un

$$\forall a, b \in A \quad a \cdot b$$

STRINGA = concatenazione di un insieme finito di caratteri (in cui l'ordine dei caratteri è

Lunghezza stringa = $|stringa|$

Operazioni stringhe = concatenazione ed esponenziale

$X^n = x * x^{(n-1)}$ --> operazione ricorsiva = **ritorsione** $X^0 = \text{stringa vuota}$ | $X^1 = X$

Una stringa è sempre prefisso, suffisso e sottostringa di se stessa

Concatenazione di due stringhe x e y

è la stringa xy ottenuta facendo seguire i simboli di y a quelli di x

Esponenziale di una stringa x

è definito da

$x^0 = \epsilon$ (ϵ stringa vuota, non contiene alcun simbolo)

$$x^n = x x^{n-1}$$

Prefisso di una stringa x

si ottiene scartando 0 o più simboli dalla sua coda

Suffisso di una stringa x

si ottiene scartando 0 o più simboli dalla sua testa

Sottostringa

si ottiene cancellando da una stringa un prefisso e un suffisso

Linguaggio

Linguaggio = insieme di stringhe formate a partire da un alfabeto

La **chiusura di Kleene *** di un

.....



Stephen Kleene
(1909 – 1999)

alfabeto

$a \cdot b = ab$

importante => while diverso da while)

Esempi

$x = \text{"programmazione"}, |x| = 14$

Prefissi: programmazione, programma, prog, ...

Suffissi: programmazione, grammazione, zione, ...

Sottostringhe: programmazione, gramma, gra, zione, ...

$x = \text{"programmazione"}, y = \text{"algoritmica"},$

$xy = \text{"programmazionealgoritmica"}, |xy| = 14+11 = 25$

$x = \text{"ab"}, |x| = 2$

$x^0 = \epsilon$

$x^1 = x = ab$

$x^2 = x x^1 = abab$

$x^3 = x x^2 = ababab$

Due linguaggi particolari sono il linguaggio vuoto $\emptyset$ e il linguaggio costituito dalla sola stringa vuota $\{\epsilon\}$

Cole Kleene
(1905-1994) è stato un

La Chiusura di Kleene si indica con *

insieme di simboli contiene tutte le stringhe di lunghezza 0, 1, 2, 3, .. che si possono formare concatenando simboli di un alfabeto



matematico
noto per essere
fondatore della
ricorsione
aiutò a sviluppare
l'informatica

ESEMPIO

se $A = \{0, 1\}$ è l'alfabeto del linguaggio dei numeri binari, A^* include tutti i possibili numeri binari, incluso il numero di lunghezza 0 che corrisponde alla stringa vuota

$\epsilon, 0, 1, 00, 01, 10, 11, 000, \dots$

2^n sequenze di lunghezza n

A^* rappresenta dunque l'insieme di tutte le stringhe sull'alfabeto A , inclusa la stringa vuota.

Se l'alfabeto A non è vuoto, A^* è un insieme infinito, formato da infinite stringhe, anche se ciascuna stringa è di lunghezza finita.

- Un linguaggio L su un alfabeto A è un sottoinsieme di A^* . La chiusura di Kleene di A : $L \subseteq A^*$
- Anche A^* è un esempio di linguaggio, così come la stringa vuota (linguaggi poco interessanti)
- I linguaggi "interessanti" su un dato alfabeto A saranno quelli che sono insiemi non vuoti e non banali di stringhe che hanno in comune qualche proprietà

co americano
essere uno dei
della teoria della
che in seguito
iluppare
ca teorica

dica tutti i
orrisponde

alfabeto

nato cioè
ezza

ne della

l'insieme

ranno
no in

Le stringhe possono essere illimitate, ma non infinite
(devono avere lunghezza finita)

A^* = insieme che contiene tutto

IT = alfabeto italiano

Chiusura di Kleene di IT = IT^* = tutte le stringhe possibili
formate dai caratteri dell'alfabeto italiano

Chiusura Positiva A^+

=> tutte le sequenze ottenibili usando i simboli, esclusa la
stringa vuota

$A = \{0,1\}$ A^+ rappresenta il binario

Linguaggio di programmazione

=> sottoinsieme di ASCII*

=> un programma è una stringa (lunga) di caratteri
alfanumerici ASCII (incluso spazio, tabulazione ecc...)

=> NON tutte le stringhe sono programmi accettabili

=> ogni ling. Di programmazione ha la propria sintassi

Linguaggio di programmazione

- si può identificare come l'insieme delle stringhe ammissibili
(che chiamiamo **programmi**)
- descrivere un LdP significa sostanzialmente descrivere
l'insieme delle stringhe del linguaggio stesso

Come identificare l'insieme che caratterizza un linguaggio?

Un **linguaggio L** su un **alfabeto A** può essere definito da tutti i suoi elementi (non praticabile per i linguaggi infiniti)

Un **linguaggio infinito** può essere definito in tre modi

1. come l'insieme delle **stringhe generate da un automa** (stringhe che seguono le regole della grammatica)

→ METODO GENERATIVO

2. come l'insieme delle **stringhe riconosciute da un automa** (strumento in grado di capire se una stringa appartiene al linguaggio)

→ METODO RICONOSCIMENTO

3. come l'insieme delle **stringhe soluzione di un sistema di equazioni algebriche**

→ METODO ALGEBRICO

le stringhe ammissibili che

finito enumerando
gi infiniti).

e modi:

na **GRAMMATICA**
matica)

=> da' un insieme finito di REGOLE
grammaticali che, se applicate, permettono di
costruire stringhe ammissibili

la un **AUTOMA**
appartiene o meno

IVO

n sistema di

(Formulazione matematica)

Se il linguaggio è composto da poche
parole è possibile definire le stringhe
ammissibili da un elenco.

GRAMMATICHE

- Notazione concisa per descrivere la sintassi di programmazione
- Sono costituite da **produzioni** (REGOLE della grammatica)
- Le grammatiche si classificano in base all'ambito del linguaggio che generano
 - **Gerarchia di Chomsky**: classificazione delle produzioni
- I linguaggi di programmazione sono definiti **libere da contesto**
Le grammatiche usate per creare linguaggi di programmazione vengono definite "libere dal contesto"



Chomsky, (1928 -), è un linguista teorico americano che negli anni '50 ha introdotto il concetto di grammatica formale



Esempio: grammatica per le espressioni

Consideriamo le **espressioni aritmetiche** che contengono

- i due operatori binari + e * Sto dando un significato ai simboli

si di un tipico linguaggio

mmatica)

espressività del

ne basata sulla forma

ti dalle **grammatiche**

guaggi di programmazione
O"

Emil Leon Post (1897 – 1954)
è stato un matematico e logico
statunitense di origine polacca
che introdusse il termine
produzione per caratterizzare la
sintassi delle teorie logiche

oni aritmetiche

no

$\rightarrow +$ indica la somma

- i due operatori binari $+$, $*$ stanno dando un significato ai simboli
- le parentesi $(,)$ $12 + 3$ o $12 + * 25$ NON ammissibili
- i numeri come operandi

Il linguaggio delle espressioni aritmetiche (stringhe che sono espressioni aritmetiche ben formate) si può definire con

4 regole che compongono la grammatica

1. *(Espressione, Numero)* => un'espressione può essere indicata da un numero
2. *(Espressione, (Espressione))* => un'espressione può essere indicata da una parentesi che racchiude un'espressione
3. *(Espressione, Espressione + Espressione)* => un'espressione può essere indicata da una somma di due espressioni
4. *(Espressione, Espressione * Espressione)* => un'espressione può essere indicata da un prodotto di due espressioni

Espressione: categoria sintattica o simbolo non terminale
qualunque stringa nel linguaggio delle espressioni aritmetiche

La notazione (A, B) significa "A può essere sostituito con B"

Numero: categoria sintattica (da interpretare come simbolo non terminale)
generica stringa che possa essere interpretata come un numero

GRAMMATICA

Una grammatica è una quadrupla $G = \langle N, \Sigma, P, S \rangle$ Insieme di 4 insiemi Simboli Non Terminali

1. $N \neq \emptyset$ è un insieme finito di **simboli non terminali**
(oggetti usati per generare le stringhe delle espressioni aritmetiche)
→ *categorie sintattiche*
2. Σ è un alfabeto di **simboli terminali**

→ indica la somma

ssibili

e corrispondono a
n la grammatica

a da un numero

sostituita così: 15 --> (15)

pressione può essere sostituita dalla
due espressioni

pressione può essere sostituita dal
di due espressioni

minale (indica una
metiche)

da B" → **PRODUZIONE**

segnaposto per una
un numero)

NON terminali

, **P, S** dove

terminali
el linguaggio)

SIMBOLI NON TERMINALI:

Categorie sintattiche = segnaposto
che usiamo per costruire le stringhe e
che vengono via via eliminati

(caratteri che compaiono nelle stringhe)

3. $P \subseteq (N \cup \Sigma)^+ \times (N \cup \Sigma)^*$ è un insieme finito (sono coppie di stringhe)
4. $S \in N$ è il **simbolo distinto** (o simbolo iniziale)

Convenzioni

$A, B, \dots \in N$ Simboli NON terminali -> indicati in MAIUSCOLO

$a, b, \dots$

$X, Y, \dots \in (N \cup \Sigma)$

$x, y, \dots$

$\alpha, \beta, \dots \in (N \cup \Sigma)^*$

$(\alpha, \beta), \dots$

ESEMPI DI GRAMMATICHE

Grammatica per le sequenze binarie

$\langle \{B\}, \{0, 1\}, \{(B, 0), (B, 1), (B, 0B), (B, 1B)\} \rangle$

B: categoria sintattica che rappresenta le stringhe binarie

Produzioni: possiamo sostituire B con 0, 1 o $0B$ o $1B$

0,1 = simboli terminali

Grammatica per le sequenze di cifre

$\langle \{D\}, \{0, \dots, 9\}, \{(D, 0), \dots, (D, 9), (D, 0D), \dots, (D, 9D)\} \rangle$

del linguaggio)

to di **produzioni**
REGOLE

iniziale)

$\dots \in \Sigma$, Simboli terminali indicati in
minuscolo

$\in \Sigma^*$,

$\alpha ::= \beta, \alpha \rightarrow \beta \in P$



—
, $B \rangle$

sequenze binarie

, $0B$ e $1B$

simboli terminali

decimali

..., $(D, 9D)\}$, $D \rangle$

$\times$ = prodotto cartesiano tra insiemi

La prima coppia deve essere NON vuota,
per questo vi è la chiusura positiva

1101 deve esser costruita partendo da B ed applica
PRODUZIONI

$B \rightarrow 1B \rightarrow 11B \rightarrow 110B \rightarrow 1101$

ando le

N Sigma

D : categoria sintattica che rappresenta le s

D = simbolo NON terminale

Produzioni: possiamo sostituire D con 0, 1

Grammatica per le espressioni aritmetici parentesi

Insieme N Insieme Sigma
 $\langle \{E\}, \{a, \dots, z, +, *, (,), \}, \{(E, a), \dots, (E, z), (E, E + E), \dots\}$
Numeri terminali

E : categoria sintattica che rappresenta le espr

Produzioni: possiamo sostituire E con $a, \dots, z,$

GERARCHIA DI CHOMSKY

Classificazione delle grammatiche in base alla t

grammatiche di tipo 0

Un unico simbolo NON Terminale può essere
sostituito da un simbolo terminale o un simbolo
terminale seguito da un simbolo NON terminale

$(\alpha, \beta), \alpha$

grammatiche dipendenti da contesto

$(\gamma A \delta, \gamma \sigma \delta), \gamma, \delta \in$

grammatiche libere da contesto

sequenze di cifre

, ..., 9, 0D, ..., 9D

che letterali con

$(E, E^* E), (E, (E))\}, E \rangle$

S simbolo distinto

ressioni

, $E + E, E^* E, (E)$

forma delle produzioni

$\alpha \in (N \cup \Sigma)^+, \beta \in (N \cup \Sigma)^*$

$\gamma \in (N \cup \Sigma)^*, \sigma \in (N \cup \Sigma)^+$

$(A \rightarrow B) \quad B \in (N \cup \Sigma)^+$

Sostituisco B con 1B / 0B

La stessa cosa è fattibile con l'insieme dei numeri n

Costruire Numero 1034 come esercizio !

GRAMMATICHE LIBERE DA CONTESTO

I linguaggi di programmazione sono definiti da **grammatiche libere da contesto** per garantire l'efficienza degli algoritmi sintattici

aturali



mmatiche
analizzatori